

CS698B: Fundamentals of Data Engineering (Part I)

2026-27-I, IIT Kanpur – Problem Sheet 3



General Instructions:

1. This problem sheet need not be submitted.
2. The questions are meant for practice and self-assessment.
3. Please use this resource honestly to assess your understanding of concepts.

Q1 (Reorder the Code). Deeborah runs a successful grocery shop and wants to build a `dict` mapping each customer to the set of distinct items purchased by them. The dict will be used to print the names of customers (in alphabetical order) who bought at least 2 distinct items. Deeborah wrote a code that works well but Deeba, eager to show off Python skills, added unnecessary line (s). Worse, Deeba recently learnt file IO and by mistake, shuffled up the lines of code in Deeborah's file. Reorder the code lines in correct order, indent properly, identify the unnecessary line(s), and print the output of the code.

```
names.sort()
for name, item, qty in records:
names = [ name for name in purchases if len( purchases[ name ] ) > 1 ]
print( names )
purchases[ name ].add( item )
purchases = {}
if name not in purchases:
purchases[ name ] = set()
purchases[ name ].append( item )
records = [ ( "bro", "bread", 2 ), ( "axe", "apple", 1 ), ( "axe", "bread", 3 ),
( "sid", "cake", 1 ) ]
```

Q2 (Reorder the Code). Deeborah decided to practice Python skills and wrote another piece of code to print the sum of squares of only even numbers from a given list. Unfortunately, Deeba got hold of this piece of code as well and added (possibly multiple) line(s) of unnecessary code. Reorder the code lines in correct order, indent properly, identify the unnecessary line(s), and print the output of the code.

```
total = sum( squares )
print( total )
squares = list( map( lambda x : x * x, evens ) )
nums = [ 1, 2, 3, 4, 5, 6 ]
evens = filter( lambda x : x % 2 == 0, nums )
evens = list( filter( lambda x : x % 2 == 0, nums ) )
squares = map( lambda x : x * x, evens )
total = sum( map( lambda x : x * x, evens ) )
```

Q3 (Become the Machine). Write the output of the following code snippet exactly, down to whitespaces.

```
def add_me( n ):
    def add( x ):
        return x + n
    return add

add3 = add_me( 3 )
add5 = add_me( 5 )
print( add3( 10 ), add5( 10 ) )

add3 = add_me( 1 )
print( add3( 10 ) )
```

CS698B: Fundamentals of Data Engineering (Part I)

2026-27-I, IIT Kanpur – Problem Sheet 3

Q4 (Become the Machine). Write the output of the following code snippet exactly, down to whitespaces.

```
def f( a, b = 2, c = 3 ):
    return a + b + c

print( f( 1 ) )
print( f( 1, 4 ) )
print( f( 1, c = 10 ) )
print( f( b = 5, a = 1, c = 2 ) )
```

Q5 (Become the Jury). Will the following two pieces of code always give the same output? If not, give a test case where their outputs differ. The input is a list of strings containing only lowercase letters.

Code 1	Code 2
<pre>def count_vowels(words): num_vowels = 0 for w in words: for c in w: if c in "aeiou": num_vowels += 1 return num_vowels</pre>	<pre>def count_vowels(words): return sum(1 for w in words for c in w if c in "aeiou")</pre>

Q6 (Become the Jury). Will the following pieces of code give the same output when the input is the list `nums = [3, 1, 9, 1, 7, 5]`? Are there other inputs where these codes will give different outputs?

SNo	Code
A	<code>result = sorted(set(nums))</code>
B	<code>result = sorted({ x for x in nums })</code>
C	<code>result = sorted(list(filter(lambda x: True, set(nums))))</code>
D	<code>result = sorted(filter(lambda x: x % 2 == 1, nums))</code>

Q7 (Rewrite the Code). Rewrite the code below using a single list comprehension. You may sort the list using the `sorted()` built-in since list comprehension need not produce sorted lists.

```
purchase_records = [ ( "axe", 81 ), ( "bro", 95 ), ( "sid", 77 ), ( "axe", 99 ) ]
high_value_customers = []

for name, bill in records:
    if bill >= 85:
        high_value_customers.append( name )

high_value_customers = sorted( set( high_value_customers ) )
print( high_value_customers )
```

Q8 (Rewrite the Code). Rewrite the code below using the built-in methods `filter()`, `map()`, `sorted()` only. No comprehension (list / set / dictionary) allowed.

```
words = [ "ox", "cat", "mouse", "me", "platypus", "ant", "bee" ]
lengths = sorted( [ len( w ) for w in words if len( w ) >= 3 ] )
print( lengths )
```